Lecture Notes: TwinCAT 3

I/O – Connecting Software to Hardware

March 17, 2026

Contents

1 Fundamentals of Industrial I/O **2**

2 EtherCAT Architecture **2**
- 2.1 EtherCAT Master . . . 2
- 2.2 EtherCAT Slave . . . 2
- 2.3 The "Train" Analogy . . . 2

3 Software to Hardware Mapping **3**

4 Practical Implementation: IO-Link Distance Sensor **5**
- 4.1 Step 1: Connecting and Scanning . . . 5
- 4.2 Step 2: Creating the Function Block . . . 6

5 Data Conversion and Bit Manipulation **7**

6 Testing and Online Monitoring **11**
- 6.1 Linking Variables to Hardware . . . 11
- 6.2 Online Monitoring Features . . . 11

Quick Reference Card **11**

1 Fundamentals of Industrial I/O

Industrial control systems interact with the physical world through **Inputs (Sensors)** and **Outputs (Actuators)**.

- **Sensors:** Gather information about the environment (e.g., temperature, flow meters, distance, humidity).
- **Actuators:** Interact with the environment based on PLC logic (e.g., motors, heaters, pumps, robots).
- **Fieldbus:** An industrial computer network used for **real-time distributed control**, connecting the PLC to various sensors and actuators. Common examples include Profinet, Modbus, and EtherCAT.

I/O Type	Direction	Examples	Data Type
Digital Input	Field → PLC	Pushbutton, Proximity sensor	BOOL
Digital Output	PLC → Field	Relay, Indicator lamp	BOOL
Analog Input	Field → PLC	Temperature sensor, Pressure	INT, REAL
Analog Output	PLC → Field	VFD speed, Valve position	INT, REAL

Table 1: Common I/O Types in Industrial Automation

Note: The distinction between inputs and outputs is from the PLC's perspective. An input brings data *into* the PLC from the field, while an output sends data *from* the PLC to field devices.

2 EtherCAT Architecture

EtherCAT is the de facto fieldbus for Beckhoff PLCs. It utilizes a **Master/Slave** architecture:

2.1 EtherCAT Master

Implemented entirely in software on the PLC; it sends out telegrams.

2.2 EtherCAT Slave

Requires a special hardware chip called an **EtherCAT Slave Controller**. This allows for extremely fast cycle times, often less than 100 microseconds.

2.3 The "Train" Analogy

An EtherCAT telegram acts like a high-speed train. It leaves the Master (PLC), stops at each Slave (station) to drop off output data and pick up sensor input data, and then returns the complete set of information back to the PLC.

Tip: EtherCAT achieves its exceptional speed by processing data "on the fly" as the telegram passes through each slave. Unlike traditional protocols where each device must receive, process, and respond to messages individually, EtherCAT slaves insert their data

Master (PLC)	→	**Slave 1** (Terminal)	→	**Slave 2** (Drive)	→
		↓			
	←	**Slave 4** (I/O Module)	←	**Slave 3** (Sensor)	←

Figure 1: EtherCAT Ring Topology (Telegram Flow)

Fieldbus	**Typical Cycle Time**	**Protocol Type**
EtherCAT	$< 100\ \mu s$	Ethernet-based
Profinet	$250\ \mu s - 1$ ms	Ethernet-based
Modbus TCP	$10 - 100$ ms	Ethernet-based
CANopen	$1 - 10$ ms	CAN-based

Table 2: Comparison of Common Industrial Fieldbuses

directly into the passing telegram.

3 Software to Hardware Mapping

To use fieldbus devices in TwinCAT, you must link the software variables to the physical hardware terminals.

- **Input Mapping:** Defined using the characters `%I*` in the variable declaration.
- **Output Mapping:** Defined using the characters `%Q*`.
- **Process Data Image:** Compiling code with these identifiers generates instance variables that can be visually linked to hardware objects in the TwinCAT I/O tree.

Example: Variable Mapping

```
// PROGRAM: IO_MappingDemo
// Demonstrates the %I* and %Q* syntax for hardware I/O mapping
// These special markers allow variables to be linked to physical I/O

PROGRAM IO_MappingDemo
VAR
    // ====================================================
    // INPUT MAPPING (%I*)
    // Variables marked with %I* can be linked to physical INPUTS
    // Data flows FROM the field device TO the PLC
    // ====================================================

    // Single boolean input (e.g., pushbutton, proximity sensor)
    bStartButton     AT %I* : BOOL;       // Digital input - start button
    bStopButton      AT %I* : BOOL;       // Digital input - stop button
    bLimitSwitch     AT %I* : BOOL;       // Digital input - limit switch

```

```
    // Array of bytes for raw sensor data (e.g., IO-Link, serial)
    // Used when a device sends multiple bytes of process data
    aRawInputData    AT %I* : ARRAY[0..3] OF BYTE;    // 4 bytes of input data

    // Analog input (typically 16-bit integer from ADC)
    nAnalogInput     AT %I* : INT;        // Analog input (e.g., 0-10V sensor)

    // Word input for 16 digital inputs from an input terminal
    wDigitalInputs   AT %I* : WORD;       // 16 digital inputs as a word

    // ====================================================
    // OUTPUT MAPPING (%Q*)
    // Variables marked with %Q* can be linked to physical OUTPUTS
    // Data flows FROM the PLC TO the field device
    // ====================================================

    // Single boolean output (e.g., relay, contactor, indicator)
    bMotorContactor AT %Q* : BOOL;       // Digital output - motor starter
    bAlarmLight     AT %Q* : BOOL;       // Digital output - alarm lamp
    bSolenoidValve  AT %Q* : BOOL;       // Digital output - pneumatic valve

    // Analog output (typically 16-bit integer to DAC)
    nSpeedReference AT %Q* : INT;        // Analog output (e.g., 0-10V to VFD)

    // Array of bytes for device commands
    aRawOutputData  AT %Q* : ARRAY[0..1] OF BYTE;    // 2 bytes of output data

    // ====================================================
    // INTERNAL VARIABLES (no mapping)
    // These are NOT linked to hardware - pure software variables
    // ====================================================
    bProcessedValue : BOOL;              // Internal logic variable
    nCalculatedSpeed: INT;               // Calculated value

END_VAR

// Program logic using mapped variables
// Once linked in the I/O tree, these update automatically each cycle

// Simple motor control logic
IF bStartButton AND NOT bStopButton AND bLimitSwitch THEN
    bMotorContactor := TRUE;
ELSIF bStopButton THEN
    bMotorContactor := FALSE;
END_IF

// Alarm if motor running but limit switch trips
bAlarmLight := bMotorContactor AND NOT bLimitSwitch;

// Scale analog input (0-32767) to speed reference
```

```
nCalculatedSpeed := nAnalogInput / 2;   // Simple scaling
nSpeedReference := nCalculatedSpeed;
```

```
// Additional examples of I/O mapping declarations

VAR
    // Multiple digital inputs as individual bits
    bSensor1          AT %I* : BOOL;
    bSensor2          AT %I* : BOOL;
    bSensor3          AT %I* : BOOL;
    bSensor4          AT %I* : BOOL;

    // Complete input terminal (e.g., EL1008 - 8 digital inputs)
    aDigitalInputs    AT %I* : ARRAY[0..7] OF BOOL;

    // Temperature input (thermocouple or RTD terminal)
    nTemperatureRaw AT %I* : INT;        // Raw ADC value

    // Encoder input (position feedback)
    nEncoderPosition AT %I* : DINT;      // 32-bit position counter

    // Status word from a drive
    wDriveStatus      AT %I* : WORD;     // Drive status bits

    // Control word to a drive
    wDriveControl     AT %Q* : WORD;     // Drive control bits

    // Setpoint to a drive
    nDriveSetpoint    AT %Q* : INT;      // Speed/torque setpoint

END_VAR
```

Warning: The `AT %I*` and `AT %Q*` syntax only marks variables as *linkable* to hardware. You must still manually link them to actual I/O in the TwinCAT System Manager by dragging from the PLC instance to the I/O terminal, or by using the "Link to..." context menu option.

4 Practical Implementation: IO-Link Distance Sensor

The lecture illustrates I/O integration using an **IFM IO-Link master** and a **laser distance sensor**. **IO-Link** is a bi-directional serial protocol for low-level sensors and actuators.

4.1 Step 1: Connecting and Scanning

1. **Edit Routes:** Establish an AMS connection to the target PLC using its IP address and administrator credentials.
2. **Scan I/O:** Use the "Scan" feature in the I/O tree to automatically detect connected EtherCAT masters and slaves.
3. **Port Configuration:** For IO-Link masters, you must manually define how many bytes of process data each port (channel) expects based on the sensor's documentation.

4.2 Step 2: Creating the Function Block

Encapsulating sensor logic in a **Function Block (FB)** is a best practice. For a distance sensor, the FB should output the calculated distance and a status bit indicating if the data is valid.

Example: Sensor Function Block Header

```
// FUNCTION BLOCK: FB_O5D100
// Encapsulates the IFM O5D100 laser distance sensor
// Converts raw IO-Link data to usable distance measurement
//
// Sensor Specifications (from IFM documentation):
// - Measurement range: 100mm to 5000mm
// - Resolution: 1mm per bit (12-bit value)
// - Process data: 2 bytes (16 bits total)
//   - Bits 15-4: Distance value (12 bits)
//   - Bit 0: "In valid range" status

FUNCTION_BLOCK FB_O5D100
VAR_OUTPUT
    // ====================================================
    // OUTPUT VARIABLES
    // These provide the processed sensor data to the caller
    // ====================================================

    // Calculated distance in millimeters
    // Valid range: 100 to 5000 mm
    nDistance        : INT;

    // Status flag indicating if measurement is valid
    // TRUE = object detected within valid measurement range
    // FALSE = object too close, too far, or no reflection
    bInValueRange    : BOOL;

    // Additional diagnostic outputs
    bSensorError     : BOOL;              // Sensor communication error
    nRawValue        : WORD;              // Raw combined word (for
   debugging)

END_VAR

VAR
    // ====================================================
    // INTERNAL VARIABLES WITH I/O MAPPING
    // These receive raw data from the IO-Link master
    // Must be linked to the EtherCAT terminal in System Manager
    // ====================================================

    // Raw input bytes from IO-Link sensor
    // Byte 0: High byte of distance + status bits
    // Byte 1: Low byte of distance
    aInputBytes      AT %I* : ARRAY[0..1] OF BYTE;

    // Alternative: Map as individual bytes
    // byHighByte    AT %I* : BYTE;       // Process data byte 0
    // byLowByte     AT %I* : BYTE;       // Process data byte 1

    // Internal working variables
```

```
    wCombinedWord    : WORD;              // Combined 16-bit value
    wDistanceRaw     : WORD;              // Extracted 12-bit distance

END_VAR
```

```
// Extended Function Block with more features

FUNCTION_BLOCK FB_O5D100_Extended
VAR_INPUT
    // Optional scaling parameters
    nScaleFactor     : INT := 1;          // Multiply distance by factor
    nOffset          : INT := 0;          // Add offset to distance
    bEnable          : BOOL := TRUE;      // Enable/disable sensor reading
END_VAR

VAR_OUTPUT
    nDistance        : INT;               // Scaled distance in mm
    nDistanceRaw     : INT;               // Unscaled distance
    bInValueRange    : BOOL;              // Valid measurement flag
    bSensorOK        : BOOL;              // Sensor communication OK
    fDistanceMeters  : REAL;              // Distance in meters
END_VAR

VAR
    // I/O Mapped variables
    aInputBytes      AT %I* : ARRAY[0..1] OF BYTE;

    // Internal
    wCombinedWord    : WORD;
    wMaskedValue     : WORD;
END_VAR
```

Bit Position	Content	Data Type	Description
15–4	Distance	12-bit UINT	Distance value in mm
3–1	Reserved	—	Not used
0	Status	BOOL	In valid range flag

Table 3: IFM O5D100 Process Data Format

Note: Always consult the sensor manufacturer's documentation for the exact process data format. The bit layout, byte order (big-endian vs. little-endian), and scaling factors vary between manufacturers and sensor models.

5 Data Conversion and Bit Manipulation

Raw data from fieldbus devices often arrives as bytes that must be manipulated to extract human-readable values. In the case of the IFM distance sensor, 12 bits of a 16-bit (2-byte) word represent the distance.

- **Explicit Conversion:** Move individual bytes into a `WORD` type using `TO_WORD`.

- **Bit Shifting:** Use `SHL` (Shift Left) to move the high-byte into the correct position.
- **Bitwise OR:** Combine the high and low bytes into a single 16-bit word.
- **Masking/Shifting:** Shift the combined word to the right (e.g., `SHR`) to align the 12 relevant bits and remove status bits.
- **Bit Accessing:** Use dot notation (e.g., `.0`) to extract individual status bits from a byte.

Example: Bit Manipulation Logic

```
// FUNCTION BLOCK: FB_O5D100 (Implementation)
// Demonstrates bit manipulation to extract sensor data
// from raw IO-Link process data bytes

FUNCTION_BLOCK FB_O5D100
VAR_OUTPUT
    nDistance        : INT;              // Distance in mm (100-5000)
    bInValueRange    : BOOL;             // TRUE if measurement valid
    nRawValue        : WORD;             // Raw 16-bit value for debugging
END_VAR

VAR
    // Raw input from IO-Link (linked to hardware)
    aInputBytes      AT %I* : ARRAY[0..1] OF BYTE;

    // Internal working variables
    wHighByte        : WORD;             // High byte converted to WORD
    wLowByte         : WORD;             // Low byte converted to WORD
    wCombinedWord    : WORD;             // Both bytes combined
    wShiftedResult   : WORD;             // After right-shift to extract distance
END_VAR

// ====================================================
// STEP 1: Convert individual bytes to WORD type
// This is necessary because bit operations work on WORD/DWORD
// TO_WORD performs a type conversion without changing the value
// ====================================================
wHighByte := TO_WORD(aInputBytes[0]);   // Convert high byte (0x00 to 0x00FF)
wLowByte := TO_WORD(aInputBytes[1]);    // Convert low byte

// ====================================================
// STEP 2: Shift the high byte left by 8 bits
// SHL(value, n) shifts 'value' left by 'n' bit positions
// Example: 0x00AB shifted left 8 = 0xAB00
// ====================================================
wHighByte := SHL(wHighByte, 8);
// If aInputBytes[0] = 0x12, wHighByte is now 0x1200

// ====================================================
// STEP 3: Combine high and low bytes using bitwise OR
// OR combines the bits: 0x1200 OR 0x0034 = 0x1234
// This reconstructs the original 16-bit sensor word
// ====================================================
```

```
wCombinedWord := wHighByte OR wLowByte;
// If bytes were [0x12, 0x34], wCombinedWord = 0x1234

// Store raw value for debugging
nRawValue := wCombinedWord;

// ====================================================
// STEP 4: Shift right to extract the 12-bit distance value
// The distance is in bits 15-4 (upper 12 bits)
// SHR(value, 4) shifts right 4 positions, discarding status bits
// Example: 0x1234 >> 4 = 0x0123 (discards lower nibble)
// ====================================================
wShiftedResult := SHR(wCombinedWord, 4);
// Result is now a 12-bit value (0 to 4095)

// Convert to INT for the output
nDistance := TO_INT(wShiftedResult);

// ====================================================
// STEP 5: Extract status bit using dot notation
// The .0 notation accesses bit 0 (LSB) of a byte or word
// Bit 0 of the low byte indicates "in valid range"
// ====================================================
bInValueRange := aInputBytes[1].0;
// TRUE if bit 0 of byte 1 is set

// Alternative using the combined word:
// bInValueRange := wCombinedWord.0;
```

```
// Additional bit manipulation examples and alternatives

VAR
    byStatusByte     : BYTE;
    wControlWord     : WORD;

    // Individual status bits
    bBit0            : BOOL;
    bBit1            : BOOL;
    bBit7            : BOOL;
    bBit15           : BOOL;

    // Masked values
    wMaskedValue     : WORD;
    byMaskedByte     : BYTE;
END_VAR

// ====================================================
// DOT NOTATION FOR BIT ACCESS
// Access individual bits of any integer type
// Bits are numbered from 0 (LSB) to n-1 (MSB)
// ====================================================
bBit0 := byStatusByte.0;     // Least significant bit
bBit1 := byStatusByte.1;     // Second bit
bBit7 := byStatusByte.7;     // Most significant bit (for BYTE)

bBit15 := wControlWord.15;   // MSB of a WORD (16-bit)

// ====================================================
```

```
// BIT MASKING WITH AND
// Use AND with a mask to isolate specific bits
// Mask: 1s in positions you want to keep, 0s elsewhere
// ====================================================
// Extract lower 4 bits (nibble)
byMaskedByte := byStatusByte AND 16#0F;      // Mask: 0000 1111

// Extract bits 11-4 (8 bits in the middle of a WORD)
wMaskedValue := wControlWord AND 16#0FF0;    // Mask: 0000 1111 1111 0000
wMaskedValue := SHR(wMaskedValue, 4);        // Shift to align

// ====================================================
// SETTING INDIVIDUAL BITS
// Use OR to set bits, AND NOT to clear bits
// ====================================================
wControlWord.0 := TRUE;      // Set bit 0
wControlWord.5 := FALSE;     // Clear bit 5

// Set bit 3 using OR with mask
wControlWord := wControlWord OR 16#0008;     // 0000 0000 0000 1000

// Clear bit 3 using AND with inverted mask
wControlWord := wControlWord AND NOT 16#0008;

// ====================================================
// COMPLETE EXAMPLE: Drive control word construction
// ====================================================
// Clear all bits first
wControlWord := 16#0000;

// Set individual control bits
wControlWord.0 := TRUE;      // Enable
wControlWord.1 := FALSE;     // Not quick stop
wControlWord.2 := TRUE;      // Enable voltage
wControlWord.3 := TRUE;      // Enable operation
// Result: wControlWord = 0x000D (binary: 0000 0000 0000 1101)
```

Operation	Syntax	Description
Shift Left	SHL(value, n)	Shift bits left by n positions
Shift Right	SHR(value, n)	Shift bits right by n positions
Rotate Left	ROL(value, n)	Rotate bits left (wrap around)
Rotate Right	ROR(value, n)	Rotate bits right (wrap around)
Bitwise AND	a AND b	1 only if both bits are 1
Bitwise OR	a OR b	1 if either bit is 1
Bitwise XOR	a XOR b	1 if bits are different
Bitwise NOT	NOT a	Invert all bits
Bit Access	var.n	Access bit n of variable

Table 4: Bit Manipulation Operations in Structured Text

Tip: When debugging bit manipulation code, use the online monitoring feature to watch both the raw input bytes and intermediate values. This helps verify that each step of the conversion is working correctly. The `nRawValue` output in the example serves this purpose.

6 Testing and Online Monitoring

Once the variables are linked to the hardware in the I/O tree, you can "Login" to the PLC to see real-time data. The lecture demonstrates this by moving the laser sensor between different objects (like a bike and a computer) and watching the `nDistance` value update instantly in the TwinCAT IDE.

6.1 Linking Variables to Hardware

1. Build the PLC project to generate the process image variables.
2. In the Solution Explorer, expand the PLC instance under "PLC" in the I/O tree.
3. Find the variables marked with `AT %I*` or `AT %Q*`.
4. Right-click and select "Link To..." or drag the variable to the corresponding terminal channel.
5. Activate the configuration and download to the PLC.

6.2 Online Monitoring Features

- **Watch values:** See real-time variable values while online.
- **Write values:** Manually change variable values for testing.
- **Force values:** Override I/O values (use with caution!).
- **Breakpoints:** Pause execution at specific lines (affects real-time!).

Warning: Forcing I/O values overrides the actual hardware state and can cause unexpected machine behavior. Always ensure safety systems are in place and personnel are clear of the machine before forcing outputs. Never force values on a production system without proper lockout/tagout procedures.

Quick Reference Card

I/O Mapping Syntax

```
// Input mapping (from field to PLC)
bInput       AT %I* : BOOL;            // Digital input
nAnalogIn    AT %I* : INT;             // Analog input
aDataIn      AT %I* : ARRAY[0..n] OF BYTE;  // Byte array input

// Output mapping (from PLC to field)
bOutput      AT %Q* : BOOL;            // Digital output
nAnalogOut   AT %Q* : INT;             // Analog output
aDataOut     AT %Q* : ARRAY[0..n] OF BYTE;  // Byte array output
```

Bit Manipulation Quick Reference

```
// Combine two bytes into a WORD
wResult := SHL(TO_WORD(byHigh), 8) OR TO_WORD(byLow);

// Extract high byte from WORD
byHigh := TO_BYTE(SHR(wValue, 8));

// Extract low byte from WORD
byLow := TO_BYTE(wValue AND 16#00FF);

// Access individual bit
bBit3 := byValue.3;

// Set a bit
wValue.5 := TRUE;

// Mask to extract bits
wMasked := wValue AND 16#0FFF;  // Keep lower 12 bits
```

Common Hex Values

Hex	Binary	Use
16#00FF	0000 0000 1111 1111	Low byte mask
16#FF00	1111 1111 0000 0000	High byte mask
16#0FFF	0000 1111 1111 1111	12-bit mask
16#000F	0000 0000 0000 1111	Low nibble mask

Last Updated: March 17, 2026